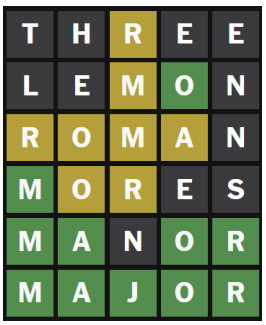
Wordle – 25 pont

A Wordle nevű internetes játékban egy szót kell kitalálni legfeljebb 6 próbálkozásból. Minden próbálkozás után jelzi az oldal, hogy a tippelt betűkből melyeket találtunk el (zöld), és melyek szerepelnek a megfejtendő szóban, csak más helyen, mint az általunk írt tippben (sárga).



Készítsd el az alkalmazás egyszerűsített változatát! A megoldás során kizárólag a **`script.js`** fájlba dolgozz, az **`index.html`** és a **`style.css`** módosítása tilos!

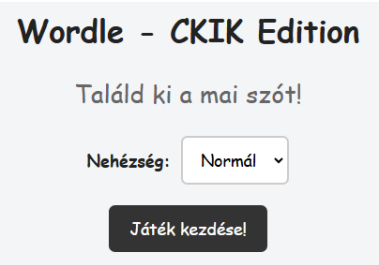
1. A `words` konstans tömb tartalmazza a lehetséges feladványokat. Készíts függvényt `getRandomWord()` néven, amely visszatérési értéke egy véletlenszerűen kiválasztott feladvány! Ügyelj rá, hogy a függvény csak nagybetűs szavakat adjon vissza! Használd a `String` osztály `toUpperCase()` metódusát! **2 pont**

   MDN: `console.log("alphabet".toUpperCase()); // 'ALPHABET'`

2. Amikor a „Játék kezdése!” gombra **kattint**unk, generálj egy random feladványt, majd tárold el a `solution` változóban! **7 pont**

   A játékos a játék nehézségét egy legördülő lista segítségével állíthatja be. A `modes` objektum tartalmazza, hogy az egyes nehézségi szinteken mennyi próbálkozási lehetőség áll rendelkezésre. Adj értéket a `countLimit` változónak is a játék kezdését követően!

   A `message` osztályú bekezdésben jelenítsd meg a minta szerint a kitalálandó szó hosszát!





   Oldd meg, hogy a játék kezdetét követően a legördülő listát tartalmazó elem ne legyen látható, viszont a játéktér (`game` azonosítójú elem) igen!

3. Írj függvényt `renderCards()` néven, amely az oldalon található számozatlan felsorolást annyi lista elemmel tölti fel, amennyi a kitalálandó szó hossza! Hívd is meg a függvényt a játék kezdetét követően! **3 pont**



4. Készíts logikai visszatérési értékű függvényt `containsChar(word, c)` néven, amely értéke pontosan akkor igaz, ha a paraméterként kapott `word` tartalmazza a paraméterként kapott `c` karaktert! A megoldáshoz tetszőleges JS függvény használható! Pl.: `containsChar("alma", "m")` **2 pont**

5. Írj függvényt `renderGuess(word)` néven, amely megjeleníti a paraméterként kapott szót a korábban generált kártyákban! **6 pont**

   Ha a szó hossza különbözik a helyes megoldás hosszától, akkor semmit nem kell tennie a függvénynek. Más esetben minden kártyára egy-egy karakter kerüljön.

   Ha egy kártyára írt karakter ugyanott szerepel, ahol a megfejtendő szóban is, a kártya háttere legyen zöld (`lightgreen`)!

   Ha a kártyán nem jó helyen szerepel, de tartalmazza a kitalálandó szó a karaktert, akkor legyen a kártya háttere sárga (`yellow`)!

   Minden más esetben a kártya háttere maradjon az eredeti fehér (`white`)!

   Pl.: ha a helyes megoldás „`TELEFON`”, akkor `renderGuess("FELADOM")` eredménye



6. A „Tippelés” gombra **kattintás**kor olvasd ki a szöveges beviteli mező tartalmát! Ha a hossza eltér a helyes megoldás hosszától, akkor az input mező kapjon `shake` stílusosztályt, és az eseménykezelő további utasításai ne fussanak le! **5 pont**

   Amennyiben megfelelő hosszúságú a játékos tippje, akkor ügyelj rá, hogy ne legyen a `shake` osztály a beviteli mezőn, továbbá rögzítsd érvényes tippként (a `count` változóban számold, hogy hány tipp volt eddig).

   Végül hívd meg a `renderGuess()` függvényt, és add át neki a játékos tippjét!

Az utolsó feladat csak **bónusz**! Jeles dolgozat esetén a bónusz feladat helyes megoldásával dupla jeles szerezhető!

7. Egy játék kétféleképpen érhet véget. Ha kitalálják a megoldást (győzelem) vagy ha elfogynak a próbálkozási lehetőségek (vereség).

   Győzelem esetén jelenítsd meg a kitalálandó szóhoz tartozó képet a `result` azonosítójú elemben! A képeket az **`images`** mappában találod, neveik megegyeznek a kitalálandó szavakkal, kiterjesztése pedig mindegyiknek `.png`!

   Vereség esetén a `result` elemben jelenjen meg, hogy vesztett a játékos, valamint a kitalálandó szó is!

   A játék végét követően a „Tippelés” gomb ne reagáljon a kattintás eseményre, és a szöveges beviteli mezőt is tiltsd le!

